

FM-Drive: Continuous Trajectory Generation without Data-Informed Priors via Flow Matching over Bernstein Coefficients for End-to-End Autonomous Driving

Anonymous Authors

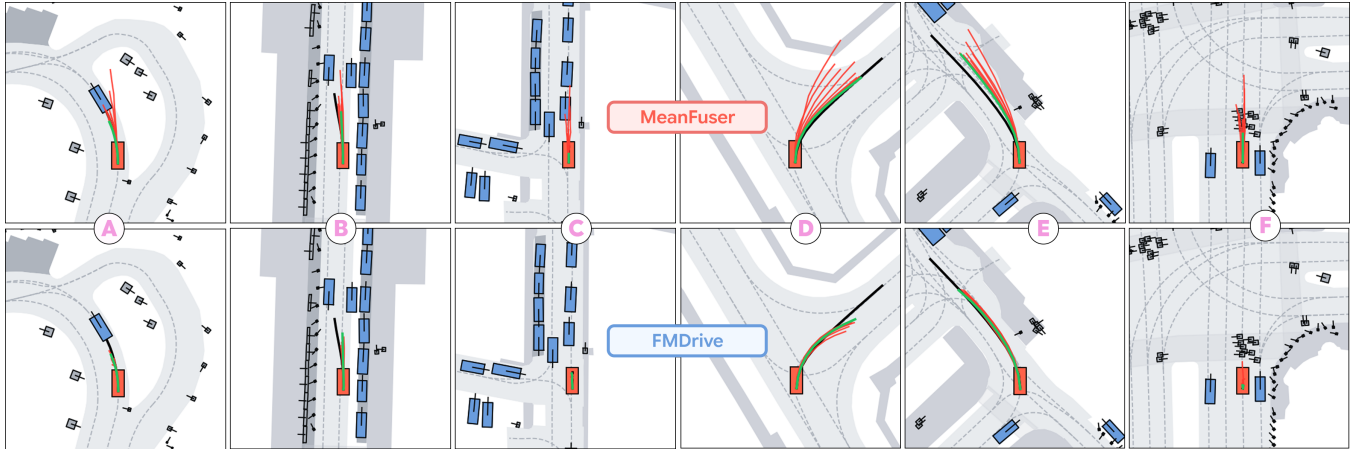


Fig. 1: **Qualitative Performance of FM-Drive against MeanFuser [1] on NAVSIMv2 open-loop benchmark.** In each panel, **red** candidates denote the trajectory proposals, **green** denotes the final aggregated trajectory passed to the controller, and **black** marks the expert log. In the roundabout scenario (A), MeanFuser’s candidates fan off the drivable corridor, whereas in the halted dense-traffic scenes (B, C), several candidates cut into parked and leading vehicles, indicating weaker consistency with the surrounding traffic configuration. At turnings (D, E), MeanFuser’s candidates spread across headings that are not feasible. In the multi-lane scene congested by pedestrians (F), candidates still predict brief corridor exits even though the vehicle is at a complete halt. FM-Drive’s candidates, however, already cluster around the feasible maneuver in every panel, remaining aligned with the drivable corridor and surrounding traffic.

Abstract—Learning-based trajectory planners for autonomous driving increasingly adopt diffusion or flow-matching formulations to model multimodal, temporally consistent action sequences. However, trajectories represented as discrete waypoints can exhibit temporally inconsistent and non-smooth motion. Top-performing baselines sidestep this by initializing the source distribution from a data-informed prior, such as clustered anchors or a Gaussian mixture fitted to the expert trajectory distribution, so that the model only has to correct a small perturbation around an already expert-like mode. While they may promote smooth trajectories, the generative process becomes biased toward modes captured by the expert-derived prior, making generation dependent on its coverage.

We propose FM-Drive, a conditional flow-matching planner that generates Bernstein polynomial coefficients conditioned on image-based scene context without using expert data in the form of anchors or mixture priors to construct its source distribution. The Bernstein parameterization ensures that every intermediate flow state decodes to a continuous, differentiable trajectory, enabling trajectory-level regularization throughout generation. We exploit this property during training through a jerk-based loss at intermediate states via endpoint reconstruction. FM-Drive further employs a DiT-based backbone with the proposed joint-attention mechanism to condition the evolving trajectory

state on perspective and bird’s-eye-view features obtained from multi-view camera-only observations. On NAVSIMv2 open-loop benchmark, FM-Drive achieves an EPDMS of 90.04, outperforming strong generative baselines without relying on data-informed structured priors.

I. INTRODUCTION

Autonomous driving requires anticipating a distribution of plausible futures while still committing to trajectories that are kinematically smooth and consumable by a downstream tracking controller. Imitation-learning planners that regress a single trajectory [2]–[5] collapse this multimodality by construction. This has motivated a shift toward generative regimes where diffusion and flow-matching-based setups [1], [6]–[10] now dominate multimodal trajectory generation.

In Flow Matching-based planners, it is common to start with isotropic Gaussian noise and to represent the target distribution as a discrete set of waypoints. However, recent top-performing generative planners in autonomous driving use source distributions that already capture substantial structure from the expert trajectory distribution. DiffusionDrive [7] perturbs a fixed vocabulary of clustered trajectory anchors

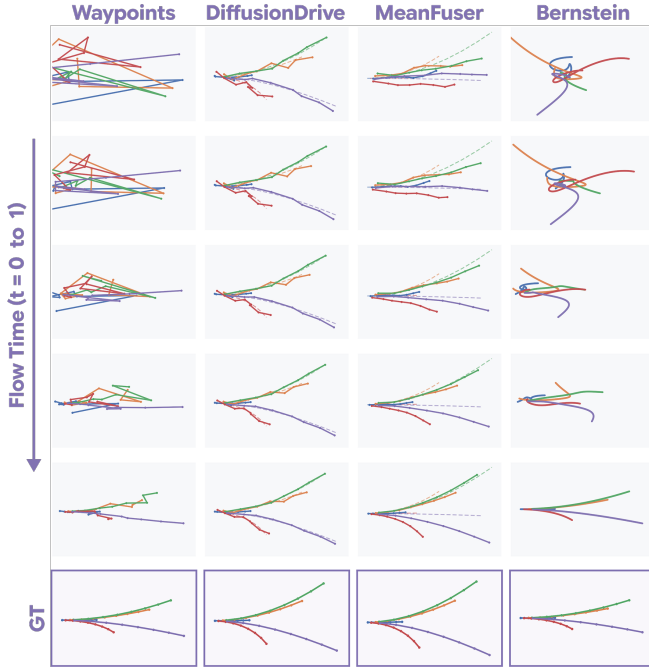


Fig. 2: **Source-to-target interpolation under different trajectory representations.** Each row shows five sampled trajectories evolving from $t = 0$ toward the expert trajectory at $t = 1$ (GT) using linear interpolation with the same targets but different starting sources. The *Waypoints* baseline starts from an isotropic Gaussian prior in discrete position space and produces highly non-smooth, self-intersecting intermediate states. *DiffusionDrive* [7] and *MeanFuser* [1] instead initialize from data-informed priors (dashed), making their early states already close to expert-like modes. In contrast, *Ours* (*Bernstein*) starts from the same uninformed isotropic prior as *Waypoints* while every intermediate state remains a smooth continuous curve by construction.

with truncated noise, while *MeanFuser* [1] fits a Gaussian mixture to the training trajectories and adds noise to the offset from the nearest component. In both cases, the model only needs to correct a small perturbation around a data-informed anchor rather than constructing the trajectory geometry from scratch, which is why the *DiffusionDrive* and *MeanFuser* columns in Fig. 2 already look clean early in the interpolation. The *Waypoints* column illustrates the motivation behind this push toward strong, data-informed priors.

We therefore argue that the output representation itself is an important part of the problem. First, discrete waypoints encode no explicit notion of the underlying trajectory geometry, so smoothness and higher-order derivatives must be inferred through finite differences between sampled points. Second, they define the trajectory only at a fixed set of discrete timesteps, requiring interpolation whenever downstream modules need states at intermediate times. Third, the representation is tied to the temporal resolution used during training, making it inconvenient for downstream components that operate at different frequencies or query the trajectory at arbitrary times. This is directly apparent in NAVSIMv2 [11],

where trajectories predicted at 2 Hz are subsequently interpolated to 10 Hz for controller-based evaluation. One alternative is to predict directly at a higher frequency, but this only increases the degrees of freedom without introducing the underlying geometry.

We present FM-Drive, a conditional flow-matching planner that overcomes these problems by changing the output space on which the generative process operates, without resorting to any data-informed source distribution. Rather than operating over noisy waypoints or expert-driven anchors, FM-Drive performs flow matching directly over the coefficients of a Bernstein polynomial. Beyond providing a compact state for flow matching, the Bernstein representation also enables trajectory-level regularization throughout the flow. The linear flow path provides a closed-form one-step reconstruction of the target coefficients from any intermediate state, while the Bernstein parameterization ensures that this reconstruction always defines a continuous, differentiable trajectory. We exploit these complementary properties to impose a flow-time-gated jerk loss on the reconstructed trajectory during training.

At its core, FM-Drive uses a Diffusion Transformer (DiT)-style backbone [12], [13] to predict the flow in Bernstein coefficient space, conditioned on complementary bird’s-eye-view and perspective-view features extracted entirely from camera observations. Instead of standard cross-attention, we introduce a *joint-attention* mechanism in which each trajectory token attends jointly to the evolving trajectory state and scene context under a shared attention distribution.

At inference, we sample multiple initial coefficient states from a standard Gaussian source and integrate the learned flow to obtain a set of Bernstein coefficient states. Each is decoded through the Bernstein basis into a continuous trajectory candidate. Since NAVSIMv2 evaluates a single trajectory per scene, we adopt the Adaptive Reconstruction Module (ARM) from *MeanFuser* [1] to aggregate these multimodal candidates into one scene-conditioned trajectory for downstream execution.

We train FM-Drive on the *navtrain* split and evaluate it on the *navtest* split of NAVSIMv2 [11]. FM-Drive achieves an EPDMS of 90.04, outperforming *DiffusionDrive* [7] and *MeanFuser* [1] by 1.9 and 0.9 points, respectively, while coming within 0.26 points of the expert reference.

In summary, FM-Drive performs conditional flow matching in the coefficient space of a Bernstein polynomial from a standard Gaussian source. A DiT-style decoder with joint attention conditions the evolving trajectory on image and BEV context, while the Bernstein representation enables flow-time-gated jerk regularization. At inference, multiple generated trajectories are aggregated by ARM into the single trajectory required by NAVSIMv2.

The contributions of this work are summarized as follows.

- We propose FM-Drive, a conditional flow-matching planner that performs generation directly in the coefficient space of a Bernstein polynomial.
- We introduce a flow-time-gated jerk regularization loss

to provide trajectory-level kinematic supervision during training through one-step endpoint reconstruction.

- We introduce a joint-attention mechanism that jointly conditions trajectory generation on the evolving trajectory state and vision features under a shared attention distribution
- FM-Drive achieves an EPDMS of 90.04 on the `navtest` benchmark, outperforming strong generative planners that rely on expert-derived structured source priors.

II. PRIOR WORK ON POLYNOMIAL REPRESENTATION IN GENERATIVE PLANNERS

We discussed the strongest driving baselines most relevant to our comparisons in the Introduction. Here we place FM-Drive in the broader context of generative planning and discuss prior work that adopts similar trajectory representations under different settings.

Parameterizing the generative state as a polynomial curve is not itself new. Motion Planning Diffusion [14] uses B-splines, whereas Guided Polynomial Diffusion (GPD) [15] denoises Bernstein coefficients, both proposed for manipulation tasks. SanD-Planner [10], denoises a B-spline trajectory representation for diffusion-based indoor navigation. In contrast, we represent the entire trajectory using a single Bernstein polynomial whose states and derivatives are defined analytically over the full planning horizon. Closest to our setting is Crowd-FM [16], which applies a similar flow-matching formulation to indoor crowd navigation. However, all of them operate in comparatively shorter-horizon, reactive settings, whereas on-road driving requires long-horizon rollout over structured interactions.

The Bernstein representation further expresses trajectory states and their derivatives as linear functions of the control points in matrix-vector form. This enables the generated trajectory to be readily incorporated into nonlinear programming or optimization-based formulation for further constraint-aware refinement. None of GPD, MPD, Crowd-FM, or SanD-Planner target on-road driving, while the driving-specific generative planners discussed so far operate on discrete trajectory representations. To our knowledge, FM-Drive is the first work to apply flow matching directly in Bernstein coefficient space for urban driving while starting from an uninformed source distribution akin to the standard flow matching setup.

III. APPROACH

In this section, we present the complete methodology for generating trajectories using flow matching conditioned on the scene and global context. We first describe the conditioning inputs fed to the time-parameterized decoder. We then introduce the polynomial trajectory representation on which the flow operates, followed by the flow matching backbone and formulation. Finally, we discuss the associated training objective and the aggregation mechanism to generate a feasible trajectory, which the downstream controller consumes during inference and evaluation. The complete FM-Drive

architecture is illustrated in Fig. 3, showing the perception and global-conditioning pathways, DiT-based flow decoder, Bernstein coefficient space, and inference procedure. We list all values and dimensions of each variable under Implementation Details IV-B to keep the presentation focused.

A. Scene Conditioning and Global Context

Perception. FM-Drive conditions the decoder on two complementary visual token sets derived from a camera-based perception stack: *Image Tokens* and *Bird’s Eye View (BEV) tokens*. Both representations are obtained from a multi-view BEV prediction model, RobustBEV [17], which we train on `navtrain` with a DINOv2 image encoder adapted via LoRA for the BEV prediction objective. We then freeze the adapted image encoder and RobustBEV branch and reuse their features as the perception backbones of FM-Drive. Both conditioning streams therefore use camera observations only, unlike baselines [7], [18], which rely on a camera-and-LiDAR setup.

Image Tokens. The current front-view image is encoded by the frozen LoRA-tuned DINOv2 backbone into a feature map $f \in \mathbb{R}^{H_f \times W_f \times C_f}$. We then flatten the feature map along its spatial dimensions (H_f, W_f) and compress it into N_x tokens using query-based attention pooling against a bank of N_x learned queries. We define the pooling operator as:

$$\text{Pool}(\mathbf{Q}, \mathbf{X}) = \text{LN}(\text{MHA}(\mathbf{Q}, \mathbf{X}, \mathbf{X}) + \mathbf{Q}) \quad (1)$$

where $\mathbf{Q}$ denotes the learned pooling queries and $\mathbf{X}$ the input feature tokens. A lightweight two-layer convolutional adapter maps the feature channels C_f to the shared perception width D_{ctx} . The resulting image representation is therefore,

$$\mathbf{Z}_{\text{img}} = \text{Pool}(\mathbf{Q}_{\text{img}}, \text{Adapter}(f)) \in \mathbb{R}^{N_x \times D_{\text{ctx}}}$$

BEV Tokens. In parallel, the frozen RobustBEV branch provides lifted features from the current multi-view cameras, using the known camera intrinsics and extrinsics, into a voxelized ground-plane representation $g \in \mathbb{R}^{H_b \times W_b \times C_b}$. We then project its channel dimension C_b to the shared perception width D_{ctx} . These features are then compressed using the same attention-pooling operator, Eq. 1, with an independent bank of N_y learned queries, yielding

$$\mathbf{Z}_{\text{bev}} = \text{Pool}(\mathbf{Q}_{\text{bev}}, \text{Proj}(g)) \in \mathbb{R}^{N_y \times D_{\text{ctx}}} \quad (2)$$

Together, $\mathbf{Z}_{\text{img}}$ and $\mathbf{Z}_{\text{bev}}$ provide complementary perspective-view and geometry-aware scene context to the decoder.

Global Context. In addition to the visual representations, FM-Drive consumes route intent through discrete driving commands (*Left, Straight, Right, Unknown* annotations provided by NAVSIMv2), the ego vehicle’s kinematic history, and a route-projected goal point.

Ego-Status. The ego history context consists of kinematic states ($v_x, v_y, a_x, a_y, \dot{\psi}$) and its footprint dimensions ($H_{\text{ego}}, W_{\text{ego}}$). We process this sequence using a three-layer MLP and use its output as the ego embedding.

Goal and Command Input. The discrete driving command and the 2D goal waypoint are independently encoded by

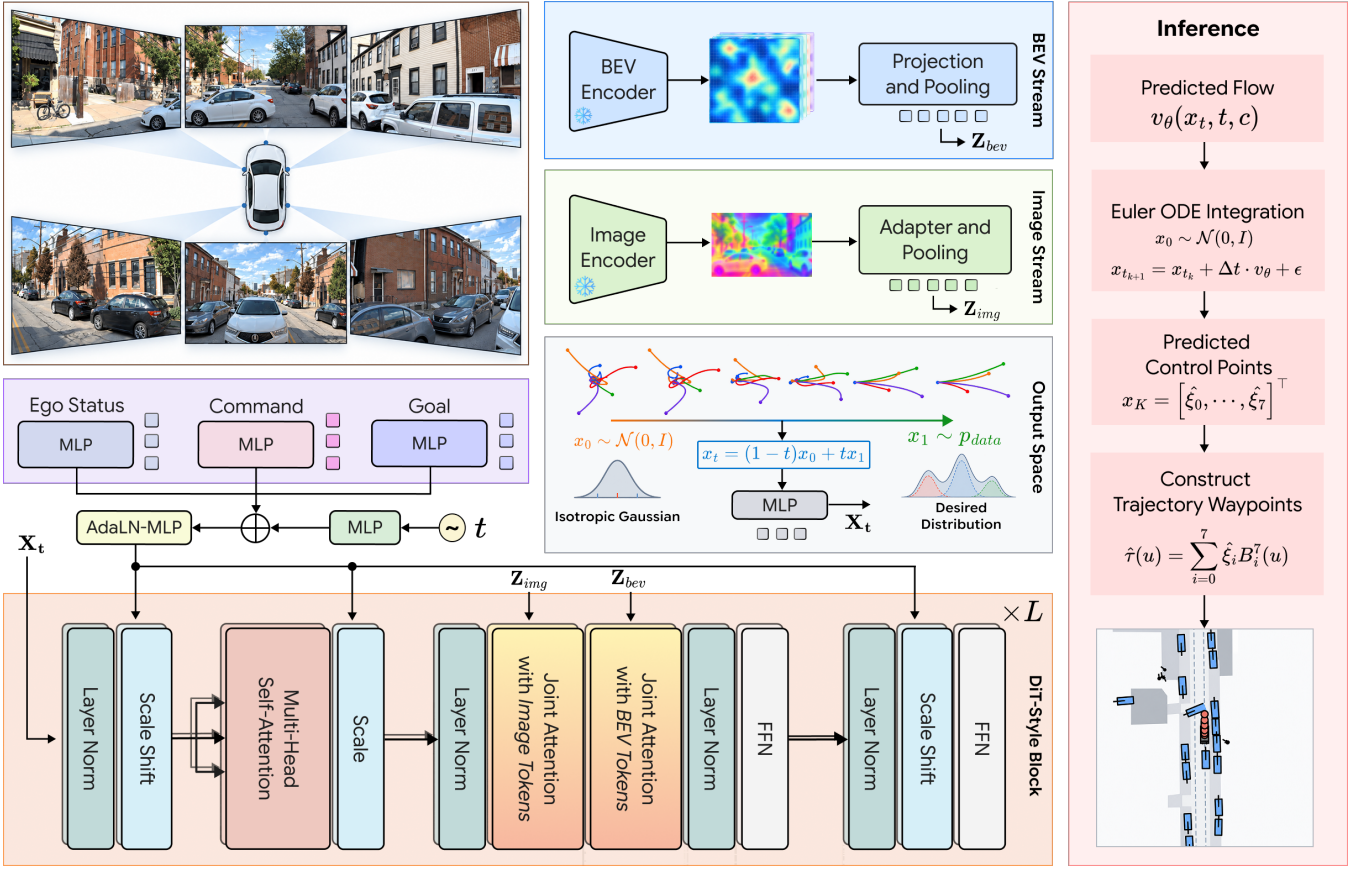


Fig. 3: **FM-Drive Architecture.** The proposed work employs a Conditional Flow Matching framework to generate scene-conditioned trajectory hypotheses, parameterized as Bernstein control points. The perception stack processes multi-view camera-only observations using frozen vision backbones, yielding token sets $\mathbf{Z}_{img}$ and $\mathbf{Z}_{bev}$. Moreover, it takes ego status and route intent (command and goal), which are pooled together with the flow-time embedding into a global context that drives AdaLN modulation. Our method adopts a DiT-style network as the Flow Matching decoder backbone, where, across L DiT blocks, the trajectory tokens and subsequently the flow become condition-aware via our proposed joint-attention mechanism. At inference, the network integrates the predicted flow v_θ over K stochastic Euler steps using Eq. (10) and maps the resulting control points through the Bernstein basis to obtain the trajectory.

lightweight two-layer MLPs into the decoder width D . The goal waypoint represents route-level intent and is treated as an external conditioning signal rather than being predicted by FM-Drive. During training and evaluation, we derive the goal location by projecting the endpoint of the expert trajectory onto the nearest lane center of the privileged HD map¹.

These global-context encoders are, however, not exposed as attendable tokens like frozen scene latents $\mathbf{Z}_{img}$ and $\mathbf{Z}_{bev}$. Section III-C describes how we incorporate them instead.

B. Trajectory Parameterization as Bernstein Control Points

Given the scene and global context, the planning objective is to generate a finite-horizon ego trajectory that is consistent with the observed scene and route intent. Following the motivation presented in the Introduction, we represent the

¹In practical autonomous driving systems, goal waypoints are typically provided by an upstream high-level planning module. The proposed Bernstein flow-matching formulation is agnostic to how this goal is obtained. In our experiments, we use privileged goal conditioning to isolate the local trajectory-generation problem from route-goal prediction.

complete trajectory using the control points of a degree- n Bernstein polynomial, so every state transported by the flow decodes to a smooth, differentiable curve.

Let $\zeta \in [0, T]$ denote physical trajectory time and define the normalized trajectory time $u = \zeta/T \in [0, 1]$. The i -th Bernstein basis function of degree n is

$$B_i^n(u) = \binom{n}{i} u^i (1-u)^{n-i}, \quad i = 0, \dots, n. \quad (3)$$

The ego trajectory is parameterized as

$$\tau(u) = \sum_{i=0}^n \xi_i B_i^n(u), \quad \xi_i \in \mathbb{R}^2, \quad (4)$$

where ξ_i is the i -th Bernstein control point. Collecting the control points gives the coefficient state $x = [\xi_0, \dots, \xi_n]^\top \in \mathbb{R}^{(n+1) \times 2}$, which is the state transported by the flow-matching process.

To convert each ground-truth waypoint trajectory into coefficient supervision, let $\mathbf{Y} \in \mathbb{R}^{m \times 2}$ contain the expert

waypoints sampled at normalized times $\{u_j\}_{j=1}^m$, and define the Bernstein design matrix $\mathbf{P} \in \mathbb{R}^{m \times (n+1)}$ by $P_{j,i} = B_i^n(u_j)$. We obtain the target coefficient state by regularized least-squares fitting:

$$x_1 = (\mathbf{P}^\top \mathbf{P} + \lambda_{\text{fit}} \mathbf{I})^{-1} \mathbf{P}^\top \mathbf{Y} \quad (5)$$

where $\lambda_{\text{fit}} = 10^{-6}$. The fitted coefficient states define the scene-conditioned target distribution $p_{\text{data}}(x | c)$, from which the target state x_1 is drawn. We use $n = 7$, yielding eight control points over the 4s planning horizon. Since both the trajectory and its derivatives are linear functions of the control points, the same coefficient representation also enables analytic trajectory-level derivative regularization. Hereafter, u denotes normalized physical trajectory time and t denotes flow-matching time.

C. Diffusion Transformer

The preceding sections defined the two decoder inputs: the Bernstein control-point state representing the trajectory and the scene plus global context that conditions its evolution. Now, at flow time t , the DiT takes this current control-point state x_t , together with these conditioning signals and predicts a velocity field v_θ in coefficient space. We parameterize v_θ using a DiT-style transformer [12] composed of L blocks with hidden width D that progressively refine the trajectory tokens. Within each block, we incorporate the visual representations sequentially through our proposed joint-attention mechanism.

The current flow-matching state $x_t \in \mathbb{R}^{(n+1) \times 2}$ in the Bernstein coefficient space is first projected to the DiT-decoder width D to get $\mathbf{X}_t \in \mathbb{R}^{(n+1) \times D}$ and is augmented with a learned positional embedding, $\mathbf{X}^{(0)} = \mathbf{X}_t W_{\text{in}} + \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(n+1) \times D}$ where W_{in} denotes the input projection and $\mathbf{X}^{(0)}$ represents the input to the first DiT block in L stacked blocks. The visual context representations $\mathbf{Z}_{\text{img}}$ and $\mathbf{Z}_{\text{bev}}$ from Section III-A are independently projected to the same decoder width before being attended to. Following the standard DiT [12], we incorporate flow time and global-context information through a pooled conditioning vector that modulates the self-attention and feed-forward branches via the AdaLN module.

Handling Shortcut Learning. Rather than passing ego-context tokens through the cross-attention block, we incorporate them into the global conditioning vector, which, together with the time embedding, drives the shift/scale/gate triplets. This design is intended to mitigate shortcut learning from ego-history information, which typically arises from biases in the data distribution [5], [19], whereby the model may over-rely on extrapolating the recent kinematic state instead of learning behaviorally meaningful scene interactions. A similar failure mode arises with goal conditioning. The model may over-optimize the trajectory endpoint to match the goal location, leading to shortcut learning [20] and potentially inducing unsafe behaviors, such as cutting across the roadway during a turn when the goal lies beyond the intersection. To reduce the tendency of the goal to act as a

direct geometric shortcut, we inject goal information through the same global conditioning pathway as the ego context.

Joint Attention. Rather than relying on standard cross-attention alone, we let each of L blocks apply what we call a *joint-attention* operation, over the current trajectory tokens $\mathbf{X}$ and context tokens $\mathbf{Z} = [\mathbf{Z}_{\text{img}}; \mathbf{Z}_{\text{bev}}]$:

$$\text{JointAttn}(\mathbf{X}, \mathbf{Z}) = \text{CrossAttn}(Q = \mathbf{X}, K = V = [\mathbf{X}; \mathbf{Z}])$$

where $[\cdot; \cdot]$ denotes concatenation along the token dimension. Trajectory tokens therefore query a key-value set containing both trajectory and scene tokens, so self-context and scene-context interactions are normalized under a single attention distribution. This lets each control-point token balance trajectory consistency against geometry-aware BEV and complementary image features. Repeating this across L decoder blocks progressively refines the coefficient state using both intra-trajectory structure and scene context.

DiT Block. For an input token sequence $\mathbf{X}$, a transformer block therefore performs:

$$\begin{aligned} \mathbf{X} &+= \alpha_1 \odot \text{MHSA}(\text{mod}(\text{LN}(\mathbf{X}), \beta_1, \gamma_1)) \\ \mathbf{X} &+= \text{JointAttn}(\mathbf{X}, \mathbf{Z}_{\text{img}}) \\ \mathbf{X} &+= \text{JointAttn}(\mathbf{X}, \mathbf{Z}_{\text{bev}}) \\ \mathbf{X} &+= \alpha_2 \odot \text{FFN}(\text{mod}(\text{LN}(\mathbf{X}), \beta_2, \gamma_2)) \end{aligned}$$

where $\text{mod}(\mathbf{x}, \beta, \gamma) = \mathbf{x} \odot (1 + \gamma) + \beta$ and $(\beta_1, \gamma_1, \alpha_1)$ and $(\beta_2, \gamma_2, \alpha_2)$ are obtained from the AdaLN layer through the global conditioning route.

Projection to v_θ . After the final L -th block, a linear output head projects the trajectory tokens back to the two-dimensional coefficient space: $\mathbf{X}^{(L)} \rightarrow v_\theta(\cdot) \in \mathbb{R}^{(n+1) \times 2}$ which represents the predicted flow at time t . The output head is zero-initialized, such that $v_\theta \equiv 0$ at the beginning of training.

D. Flow Matching

FM-Drive learns a conditional transport from the standard Gaussian source $p_0 = \mathcal{N}(0, I)$ to the scene-conditioned expert coefficient distribution. Given scene context c , we independently sample $x_0 \sim p_0$ and $x_1 \sim p_{\text{data}}(x | c)$, yielding the product coupling,

$$p(x_0, x_1 | c) = p_0(x_0) \cdot p_{\text{data}}(x_1 | c).$$

We define the linear probability path:

$$x_t = (1 - t)x_0 + tx_1, \quad t \sim \mathcal{U}[0, 1] \quad (6)$$

whose conditional target velocity is $u_t = x_1 - x_0$. The decoder v_θ is then trained to regress this velocity field from the intermediate state x_t at flow time t , given the corresponding scene and global context.

E. Training Objective and Inference

We train FM-Drive with a flow-matching regression objective and an auxiliary jerk loss. For flow matching, we use a Huber loss, denoted by $\rho(\cdot)$, between the predicted velocity $v_\theta(x_t, t, c)$ and the conditional target velocity $u_t = x_1 - x_0$:

$$\mathcal{L}_{FM} = \mathbb{E}_{c, x_0, x_1, t} [\rho(v_\theta(x_t, t, c) - u_t)] \quad (7)$$

Jerk Regularization. Rather than applying it directly to the intermediate state x_t , we use the current flow prediction $v_{\theta,t}$ to estimate the coefficient state at $t = 1$ through a one-step projection:

$$\hat{x}_1 = x_t + (1 - t) \cdot v_{\theta}(x_t, t, c) \quad (8)$$

We then compute the trajectory jerk from the estimated Bernstein control points as $\mathbf{B}^{(3)}\hat{x}_1$, where $\mathbf{B}^{(3)}$ denotes the third derivative of the Bernstein basis with respect to normalized trajectory time u .

However, the reliability of the endpoint estimate varies along the flow. In particular, its estimation error satisfies: $\hat{x}_1 - x_1 = (1 - t)(v_{\theta}(x_t, t, c) - u_t)$, such that velocity-prediction errors have a larger effect on the reconstructed trajectory at early flow times. Moreover, x_t remains more strongly influenced by the Gaussian source for small t , while as $t \rightarrow 1$ the trajectory structure becomes increasingly well formed. We therefore weight the jerk regularization by t^{α_j} evaluated at $N_j = 8$ uniformly spaced temporal samples:

$$\mathcal{L}_{\text{jerk}} = \mathbb{E}_{c,t,x_0,x_1} \left[t^{\alpha_j} \frac{1}{N_j} \left\| \mathbf{B}^{(3)}\hat{x}_1 \right\|_F^2 \right] \quad (9)$$

where α_j is the flow-time weighting exponent, set to 0.3, thereby emphasizing jerk supervision toward later stages of the flow, where the reconstructed trajectory provides a more reliable estimate of the target trajectory. Therefore, the total training loss is:

$$\mathcal{L} = \mathcal{L}_{FM} + \lambda_{\text{jerk}} \mathcal{L}_{\text{jerk}}$$

where λ_{jerk} is scheduled according to a staged, epoch-based curriculum increasing from 0.01 to 0.5 in discrete steps.

Inference. We sample $x_0 \sim \mathcal{N}(0, I)$ and integrate the learned flow using stochastic Euler integration:

$$x_{k+1} = x_k + \Delta t \cdot v_{\theta}(x_k, k\Delta t, c) + \eta(1 - k\Delta t)\sqrt{\Delta t} \epsilon_k, \quad (10)$$

where, $\Delta t = 1/K$, $\epsilon_k \sim \mathcal{N}(0, I)$. Here, $\eta \geq 0$ controls the magnitude of inference-time stochasticity, which we set to 0.1. Setting $\eta = 0$ recovers explicit-Euler integration of the learned flow ODE. After K integration steps, we map the final coefficient state x_K into the continuous trajectory using the Bernstein basis, Eqs. (3) and (4), which is then used by the NAVSIMv2 simulator for evaluation.

F. Candidate Aggregation via Adaptive Reconstruction

During inference, we sample multiple trajectory proposals, but NAVSIMv2 evaluates only a single trajectory per scene, executed open-loop by a downstream tracking controller. We therefore adopt the Adaptive Reconstruction Module (ARM) from MeanFuser [1] to aggregate the N_{cand} candidates into a single scene-conditioned trajectory. Each candidate $\hat{\tau}_i$ is encoded by an MLP to obtain its global embedding. The resulting set of N_{cand} candidate embeddings is passed through a transformer decoder that cross-attends to the BEV context $\mathbf{Z}_{\text{bev}}$, allowing ARM to aggregate the candidates according to their consistency with the surrounding scene rather than treating all samples uniformly. The fused representation is then passed through a projection head that directly regresses

the final trajectory $\hat{\tau}$. We train ARM in a post-training stage using an ℓ_1 reconstruction loss with respect to the expert trajectory τ :

$$\mathcal{L}_{\text{ARM}} = \|\tau - \hat{\tau}\|_1$$

IV. EXPERIMENTS AND RESULTS

A. Dataset and Metrics

We train on `navtrain` and evaluate on the `navtest` split of the NAVSIMv2 benchmark [11], which is built upon the nuPlan dataset [21]. NAVSIMv2 measures overall driving performance using the Extended Predictive Driver Model Score (EPDMS) metric, which is a weighted combination of nine sub-metrics: No-at-fault Collisions (NC), Drivable Area Compliance (DAC), Driving Direction Compliance (DDC), Traffic Light Compliance (TLC), Ego Progress (EP), Time-to-Collision (TTC), Lane Keeping (LK), History Comfort (HC), and Extended Comfort (EC), used in Table I.

A Critical Nuance: Crucially, the `navtest` evaluation pipeline *does not* compute EPDMS directly on the raw predicted waypoints. Instead, it uses a PDM-style tracking controller which executes the predicted trajectory open-loop against a fixed, non-reactive background of logged agents. However, because the evaluation is open-loop, for most scenes in `navtest`, faithfully replicating the expert’s kinematic profile offers a more direct path to a high score than generating genuinely diverse but plausible alternatives.

To validate this, we propose an ablation study (denoted *ADE Oracle* in Table II) where we sample multiple trajectory candidates and report the score only for the one closest to the expert ground truth. This quantifies how much performance can be recovered by selecting candidates according to their proximity to the logged expert trajectory. Section IV-D elaborates on the results.

CL-Proxy. EPDMS aggregates safety, progress, compliance, and comfort-related terms. Since our Bernstein representation and jerk regularization directly target trajectory smoothness, we additionally report CL-Proxy [22] to test whether the observed improvement persists when comfort-related terms are excluded:

$$\text{CL-Proxy} = 100 (\text{NC} \times \text{DAC} \times \text{EP})$$

where NC, DAC, and EP are normalized to $[0, 1]$. We use CL-Proxy also as a complementary indicator of whether the open-loop gains of FM-Drive remain favorable under a metric empirically associated with closed-loop performance.

B. Implementation Details

All our training and experiments are performed on a single NVIDIA RTX PRO 6000 GPU with 96 GB VRAM. We train for 50 epochs using AdamW with an initial learning rate of 2×10^{-4} , cosine decay, weight decay of 0.1, and a 3-epoch warm-up. The network plans a trajectory for a 4-second horizon at 2 Hz, and is parameterized with 8 control points using a degree-7 Bernstein polynomial. Image and BEV features of size $H_f \times W_f = 16 \times 28$ and $H_b \times W_b = 200 \times 200$ are pooled to $N_x = 112$ and $N_y = 256$ tokens, both projected to a shared perception width $D_{ctx} = 256$.

TABLE I: **Performance on the NAVSIMv2 navtest benchmark.** [†] denotes our evaluation with the official checkpoint. **Bold** denotes the best learned-planner result; Human Agent is an expert reference.

Method	NC [†]	DAC [†]	DDC [†]	TLC [†]	EP [†]	TTC [†]	LK [†]	HC [†]	EC [†]	EPDMS [†]
Human Agent	100.0	100.0	99.8	100.0	87.4	100.0	100.0	98.1	90.1	90.3
Ego Status MLP [5]	93.1	77.9	92.7	99.6	86.0	91.5	89.4	98.3	85.4	64.0
TransFuser [2]	96.9	89.9	97.8	99.7	87.1	95.4	92.7	98.3	87.2	76.7
Hydra-MDP++ [23]	97.2	97.5	99.4	99.6	83.1	96.5	94.4	98.2	70.9	81.4
DriveSuprim [18]	97.5	96.5	99.4	99.6	88.4	96.6	95.5	98.3	77.0	83.1
PRIX [24]	98.0	95.6	99.5	99.8	87.4	97.2	97.1	98.3	87.6	84.2
DiffusionDrive [†] [7]	98.2	96.3	99.4	99.8	87.4	97.4	97.0	98.3	87.7	88.1
MeanFuser [†] [1]	98.3	97.2	99.6	99.8	87.6	97.4	97.3	98.3	88.2	89.1
FM-Drive (Ours)	99.4	96.7	99.8	99.9	87.3	98.6	98.3	98.2	87.9	90.04

The DiT decoder uses $L = 12$ blocks, width $D = 64$ and 8 attention heads. At inference, we draw 8 proposals, integrate the learned flow using the stochastic Euler update in Eq. (10) over $K = 10$ steps, and aggregate them using the learned ARM module to generate the final trajectory. The resulting navtest performance is reported in Table I.

C. Main Results

Table I compares FM-Drive with deterministic [2], [18], [23] and generative [1], [7], [24] planners on navtest. The recent generative baselines already operate close to the expert reference. Our evaluations of the official DiffusionDrive and MeanFuser checkpoints obtain 88.1 and 89.1 EPDMS, respectively. Note that the Human Agent reports 90.3 score. Thus, only 1.2 EPDMS points separate the strongest reproduced baseline from the expert reference.

FM-Drive achieves **90.04 EPDMS**, improving over DiffusionDrive and MeanFuser by 1.94 and 0.94 points, respectively, and reducing the remaining gap to the Human Agent to only 0.26 points. Relative to MeanFuser, this closes approximately 78% of the remaining EPDMS gap. FM-Drive also obtains the strongest learned-planner results in NC (99.4), DDC (99.8), TLC (99.9), TTC (98.6), and LK (98.3), while remaining comparable in DAC, EP, HC, and EC. In particular, HC and EC remain nearly unchanged relative to the strongest baselines, indicating that the EPDMS improvement is not explained simply by higher comfort scores.

D. Ablations

Component Ablation. Table II isolates the contribution of each component. All FM-Drive variants use $N_{\text{cand}} = 8$, while the reproduced baselines retain their native candidate counts. Replacing waypoint generation ($\mathcal{M}_0$) with Bernstein coefficients ($\mathcal{M}_1$) raises EPDMS from 72.0 to 83.2 (+11.2). Flow-time-gated jerk regularization ($\mathcal{M}_2$) further raises it to 89.6 (+6.4), while ARM produces the final 90.04 (+0.44). Thus, most of the gain arises before aggregation, from the Bernstein representation and the trajectory-level regularization it enables.

CL-Proxy Analysis. The same trend is visible under the complementary closed-loop proxy metric. Bernstein gener-

TABLE II: **Component ablations and ADE-based candidate-selection analysis.** N_{cand} denotes the number of generated candidates. DD and MF denote DiffusionDrive [7] and MeanFuser [1], respectively. CL-Proxy complements EPDMS using NC, DAC, and EP. The ADE-based rows select the generated candidate closest to the logged expert trajectory.

Model	Components	N_{cand}	CL-Proxy [†]	EPDMS [†]
DD [7]		20	82.6	88.1
MF [1]		8	83.7	89.1
Base	FM-Drive Backbone			
$\mathcal{M}_0$	Base + Waypoints	8		72.0
$\mathcal{M}_1$	Base + Bernstein	8	77.0	83.2
$\mathcal{M}_2$	$\mathcal{M}_1$ + Jerk Loss	8	82.8	89.6
$\mathcal{M}_3$	$\mathcal{M}_2$ + ARM	8	84.0	90.04
	$\mathcal{M}_2$ + ADE	8	84.6	90.8
ADE Oracle	DD + ADE	20	83.4	89.1
	MF + ADE	8	84.4	89.6

ation obtains 77.0, jerk regularization raises it to 82.8, and ARM reaches 84.0. The full FM-Drive model also exceeds MeanFuser (83.7) and DiffusionDrive (82.6). Hence, FM-Drive’s improvement on CL-Proxy indicates that its gains are not specific to EPDMS and remain visible under a metric shown to correlate with closed-loop performance ranking.

Candidate-Set Quality. Finally, we examine whether the remaining performance is primarily limited by candidate generation or by downstream aggregation. For each method, we use an ADE-based oracle that selects the generated candidate closest to the logged expert. Under this common selector, FM-Drive reaches **90.8 EPDMS / 84.6 CL-Proxy**, compared with 89.6/84.4 for MeanFuser and 89.1/83.4 for DiffusionDrive. Notably, FM-Drive with learned ARM already achieves 90.04 EPDMS, exceeding the ADE-selected scores of both baselines, indicating that the advantage lies in the generated candidate set rather than arising solely from ARM.

Interestingly, the ADE-selected FM-Drive candidate reaches 90.8 EPDMS, slightly above the 90.3 Human Agent reference. This should not be interpreted as super-human driving: the selector uses privileged expert information, and

the Human Agent serves as a benchmark reference rather than a formal upper bound on EPDMS. Instead, the result shows that the generated candidate set contains trajectories that score at or above the logged expert reference under the NAVSIMv2 metric. The remaining 0.76-point gap between ADE selection and ARM suggests additional headroom in candidate aggregation.

V. CONCLUSION

We presented FM-Drive, a conditional flow-matching planner operating directly in Bernstein coefficient space from a standard Gaussian source, unlike current top-performing baselines which use expert-driven priors. The Bernstein representation provides a smooth, differentiable trajectory at every flow state, enabling intermediate-state jerk regularization. Our ablations show substantial gains from Bernstein generation (+11.2 EPDMS) and jerk regularization (+6.4), with ARM producing the final 90.04 EPDMS. FM-Drive also reaches 84.0 CL-Proxy, outperforming DiffusionDrive and MeanFuser on both reported metrics.

Limitations: Our evaluation is still open-loop. NAVSIMv2 re-simulates a single predicted trajectory against a fixed, non-reactive background, so genuine closed-loop behavior remains untested. Moreover, our experiments also use privileged route-goal conditioning to isolate local trajectory generation, so evaluating route goals supplied by a non-privileged upstream planner is a direction to explore. Finally, our decoder requires multi-step flow integration at inference, incurring greater sampling cost than one-step planners such as MeanFuser; reducing this cost without sacrificing trajectory quality remains future work.

REFERENCES

- [1] J. Wang, Y. Zheng, X. Liu, Z. Xing, P. Li, K. Ma, H. Ye, G. Chen, G. Li, L. Chen, *et al.*, “Meanfuser: Fast one-step multi-modal trajectory generation and adaptive reconstruction via meanflow for end-to-end autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 17 884–17 893.
- [2] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger, “Transfuser: Imitation with transformer-based sensor fusion for autonomous driving,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11, pp. 12 878–12 895, 2022.
- [3] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, *et al.*, “Planning-oriented autonomous driving,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 17 853–17 862.
- [4] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang, “Vad: Vectorized scene representation for efficient autonomous driving,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 8340–8350.
- [5] Z. Li, Z. Yu, S. Lan, J. Li, J. Kautz, T. Lu, and J. M. Alvarez, “Is ego status all you need for open-loop end-to-end autonomous driving?” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 2024, pp. 14 864–14 873.
- [6] C. M. Jiang, A. Cornman, C. Park, B. Sapp, Y. Zhou, and D. Anguelov, “Motiondiffuser: Controllable multi-agent motion prediction using diffusion,” in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 2023, pp. 9644–9653.
- [7] B. Liao, S. Chen, H. Yin, B. Jiang, C. Wang, S. Yan, X. Zhang, X. Li, Y. Zhang, Q. Zhang, *et al.*, “Diffusiondrive: Truncated diffusion model for end-to-end autonomous driving,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 12 037–12 047.
- [8] Z. Xing, X. Zhang, *et al.*, “Goalflow: Goal-driven flow matching for multimodal trajectories generation in end-to-end autonomous driving,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 1602–1611.
- [9] L. Liu, C. Jia, G. Yu, Z. Song, J. Li, F. Jia, P. Wu, X. Hao, and Y. Luo, “Guidelflow: Constraint-guided flow matching for planning in end-to-end autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 3719–3728.
- [10] J. Wang, L. Bao, T. Yang, D. M. Plasencia, J. Jiao, and D. Kanoulas, “SanD-Planner: Sample-Efficient Diffusion Planner in B-Spline Space for Robust Local Navigation,” in *Proceedings of Robotics: Science and Systems*, Sydney, Australia, July 2026.
- [11] W. Cao, M. Hallgarten, T. Li, D. Dauner, X. Gu, C. Wang, Y. Miron, M. Aiello, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, “Pseudo-simulation for autonomous driving,” in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Lim, S. Song, and H.-W. Park, Eds., vol. 305. PMLR, 27–30 Sep 2025, pp. 4709–4722. [Online]. Available: <https://proceedings.mlr.press/v305/cao25a.html>
- [12] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2023, pp. 4172–4182.
- [13] J. Chen, J. Yu, C. Ge, L. Yao, E. Xie, Z. Wang, J. T. Kwok, P. Luo, H. Lu, and Z. Li, “Pixart- α : Fast training of diffusion transformer for photorealistic text-to-image synthesis,” in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=eAKmQPe3m1>
- [14] J. Carvalho, A. T. Le, P. Kicki, D. Koert, and J. Peters, “Motion planning diffusion: Learning and adapting robot motion planning with diffusion models,” *IEEE Transactions on Robotics*, vol. 41, pp. 4881–4901, 2025.
- [15] A. Srikanth, P. Mahajan, K. Saha, V. Mandadi, P. Paul, P. Wadhvani, B. Bhowmick, A. Singh, and M. Krishna, “Gpd: Guided polynomial diffusion for motion planning,” in *2025 IEEE 21st International Conference on Automation Science and Engineering (CASE)*. IEEE, 2025, pp. 2758–2765.
- [16] A. Singha, L. Nanwani, S. Jain, P. T. Singamaneni, A. K. Singh, K. M. Krishna, *et al.*, “Crowd-fm: Learned optimal selection of conditional flow matching-generated trajectories for crowd navigation,” *arXiv preprint arXiv:2602.06698*, 2026.
- [17] M. R. Barin, G. Aydemir, and F. Güney, “Robust bird’s eye view segmentation by adapting dinov2,” *arXiv preprint arXiv:2409.10228*, 2024.
- [18] W. Yao, Z. Li, S. Lan, Z. Wang, X. Sun, J. M. Alvarez, and Z. Wu, “Drivesuprim: Towards precise trajectory selection for end-to-end planning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 14, 2026, pp. 11 910–11 918.
- [19] B. Jaeger, K. Chitta, and A. Geiger, “Hidden biases of end-to-end driving models,” in *2023 IEEE/CVF international conference on computer vision (ICCV)*. IEEE, 2023, pp. 8206–8215.
- [20] L. Nguyen, M. Fauth, B. Jaeger, D. Dauner, M. Igl, A. Geiger, and K. Chitta, “Lead: Minimizing learner-expert asymmetry in end-to-end driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 39 775–39 785.
- [21] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari, “nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles,” *arXiv preprint arXiv:2106.11810*, 2021.
- [22] Y. Wang, A. Jiang, S. Wang, Y. Heng, H. Yang, Y. Chen, and H. Sun, “Do open-loop metrics predict closed-loop driving? a cross-benchmark correlation study of navsim and bench2drive,” *arXiv preprint arXiv:2605.00066*, 2026.
- [23] K. Li, Z. Li, S. Lan, Y. Xie, Z. Zhang, J. Liu, Z. Wu, Z. Yu, and J. M. Alvarez, “Hydra-mdp++: Advancing end-to-end driving via expert-guided hydra-distillation,” *arXiv preprint arXiv:2503.12820*, 2025.
- [24] M. Wozniak, L. Liu, Y. Cai, and P. Jensfelt, “Prix: Learning to plan from raw pixels for end-to-end autonomous driving,” *IEEE Robotics and Automation Letters*, vol. 11, no. 5, pp. 6400–6407, 2026.